3.28-3.31:

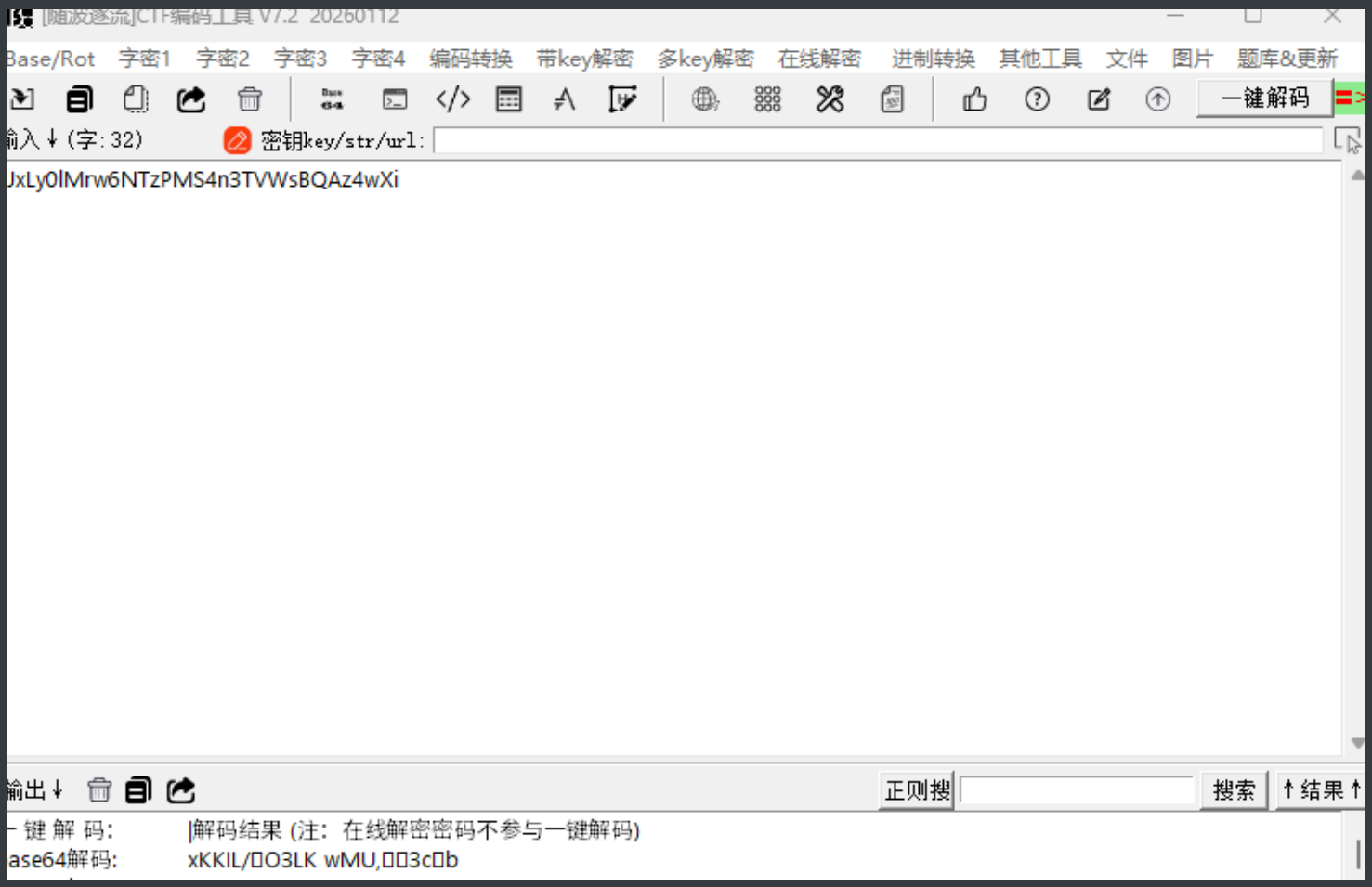
1、3-track_hacker-Bugku：

下载后得到一个pcap的流量包对其进行流量分析先查找一下flag的字眼查到发现一个flag.txt的字眼，追踪一下tcp流可以看到

Wireshark · 追踪 TCP 流 (tcp.stream eq 3) · shell.pcap

```
POST /.config.php HTTP/1.1
Host: www.cometohackme.com
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.14; rv:48.0) Gecko/20100101 Firefox/48.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: zh-CN,zh;q=0.8,en-US;q=0.5,en;q=0.3
Accept-Encoding: gzip, deflate
Connection: keep-alive
Upgrade-Insecure-Requests: 1
Content-Type: application/x-www-form-urlencoded
Content-Length: 30

k=cometohackme&c=cat /flag.txt
HTTP/1.1 200 OK
Date: Mon, 09 Jul 2018 03:22:10 GMT
Server: Apache/2.4.18 (Ubuntu)
Content-Length: 32
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/html; charset=UTF-8

eJxLy0lMrw6NTzPMS4n3TVWsBQAz4wXi
```

当时以为要将这个flag.txt的文件弄出来，尝试了导出一下文件但是什么都没发现，后面才发现最后一行有一个类似base编码的密文，我们放到随波逐流里面进行解密但输出的是乱码然后就没有头绪了看了一下bugku的评论区说还要进行解压处理



进行解密及解压的python代码如下：

```python
import base64
import zlib
flag = "eJxLy0lMrw6NTzPMS4n3TVWsBQAz4wXi"
decoded = base64.b64decode(flag)
result = zlib.decompress(decoded)
print(result)
```

运行后就可以得到我们的flag：b'flag{U_f1nd_Me!}'

2、3-Not_only_base-Bugku:

下载后得到一个无后缀的文件老样子先改成txt看能不能阅读改完后里面的内容为：
MCJIJSGKP=ZZYXZXRMU=W3YZG3ZZ==G3HQHCUS==

这个很明显是某种编码然后又有等号的话只有base类型的，但是base类型的等号都是在末尾的，想到栅栏密码可以改变字符串的顺序，先将原始的密文放到随波逐流里面进行栅栏解密找到等号都在最后的解密结果

```
栅栏fence解码:
因数[2, 4, 5, 8, 10, 20]:
分为2栏时，解密结果为:MWC3JYIZJGS3GZKZP==ZGZ3YHXOZHXCRUMSU==
分为4栏时，解密结果为:MZWGCZ33JYYHIXZQJZGHSX3CGRZUKMZSPU======
分为5栏时，解密结果为:MPRGHC=M3QJZUZHIZ=ZCJYW=USX3=SGZYG=KXZ3=
```

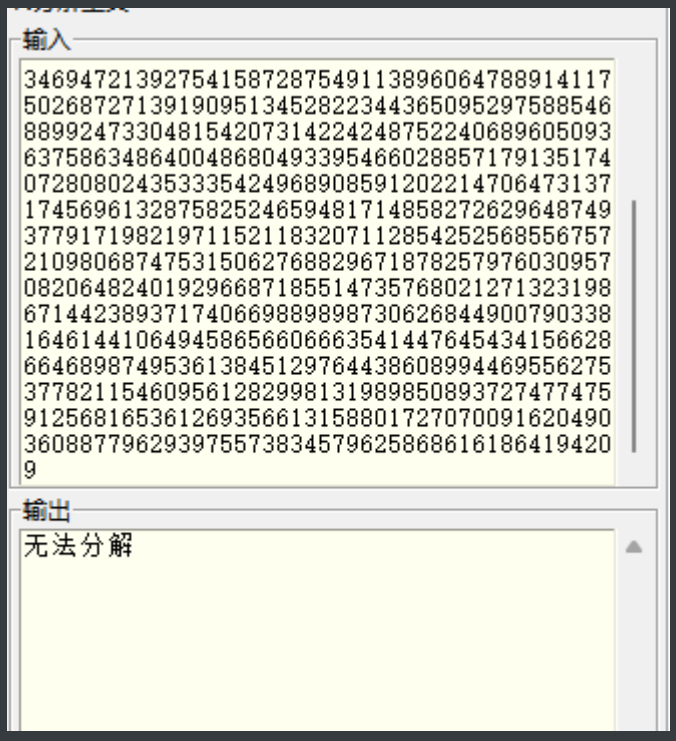
然后再进行解密就可以得到flag：flag{N0t_0NLy_b4sE32}

3、RSA2-Bugku：

下载后得到RSA算法的几个重要参数

```
e = 2
N =
50045694405121100953159124242361752742185500021785206285537046101881814883001077579187269938169251075763588392072383920327242797123579446009811504180443429049551832465449201545280202036472287060805906393403437359823162117030733375749813508457540166385619337652808846785706803675035681409335150834694721392754158728754911389606478891411750268727139190951345282234436509529758854688992473304815420731422424875224068960509363758634864004868049339546602885717913517407280802435333542496890859120221470647313717456961328758252465948171485827262964874937791719821971152118320711285425256855675721098068747531506276882967187825797603095708206482401929668718551473576802127132319867144238937174066988989873062684490079033816461441064945865660666354144764543415662866468987495361384512976443860899446955627537782115460956128299813198985089372747747591256816536126935661315880172707009162049036088779629397557383457962586861618641942 09)

the encrypted flag =
17857805411358766588057363852554397135782412262215950548872038436860295318116656453473068060315237682513079441939371616424714351536106697137940979086258425012 1
```

看到e=2知道了这是道低指数加密的题目先看看能不能分解N



看来这个N没办法分解，那就尝试直接对密文进行开平方处理对应的py代码如下：

```python
import gmpy2
c =
17857805411358766588057363852554397135782412262215950548872038436860295318116656453473
06806031523768251307944193937161642471435153610669713794097908625842501 21
m = gmpy2.isqrt(c)
if m*m == c:
    print("m =", m)
    print("hex =", hex(m))
    print("bytes =", bytes.fromhex(hex(m)[2:]))
else:
    print("not exact square")
```

运行后果然找到了我们的flag：

```
E:\pythonproject\.venv\Scripts\python.exe E:\pythonproject\CTF常用算法\temp.py
m = 13363309998409363618670954816880563712066497261308319592953169265167568993531261
hex = 0x7368656c6c6d617465737b34763069645f356d346c6c5f337870306e336e74357d
bytes = b'shellmates{4v0id_5m4ll_3xp0n3nt5}'
```

4、source-Bugku：

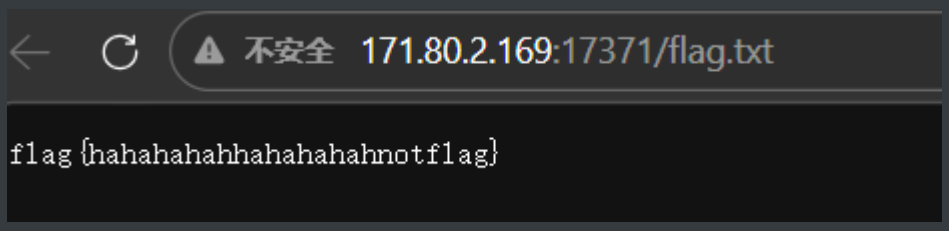
打开靶机后按f12查看一下源码，发现有个flag：flag{Zmxhz19ub3RfaGvyzSEHIQ==}直接提交显然不对，里面还有个类似base编码的密码，先进行一下解密看一下

一键解码:	解码结果 (注：在线解密密码不参与一键解码)
base64解码:	flaO_not_hkrM!□!

看样子这个flag根本就不是正确答案，先扫个目录看一下

```
PS E:\CTF工具\CTF资料包\Web\扫描工具\dirsearch-master> python .\dirsearch.py -u http://171.80.2.169:17371/ -x 503

  _|. _ _  _  _  _ _|_    v0.4.3
 (_||| _) (/_(_|| (_| )

Extensions: php, asp, aspx, jsp, html, htm | HTTP method: GET | Threads: 25 | Wordlist size: 12295

Target: http://171.80.2.169:17371/

[20:42:40] Scanning:
[20:42:41] 301 -   320B - /.git  ->  http://171.80.2.169:17371/.git/
[20:42:42] 403 -   280B - /.php
[20:42:47] 200 -    32B - /flag.txt
[20:42:48] 200 -   139B - /index.html
[20:42:50] 403 -   280B - /server-status
[20:42:50] 403 -   280B - /server-status/

Task Completed
PS E:\CTF工具\CTF资料包\Web\扫描工具\dirsearch-master>
```

找到一个flag.txt的文件和一个git目录，先看一下txt中有没有东西



呃好吧又被耍了这个不是flag那就是git泄露了，去Kail里面修复一下git目录，先用wget命令下载全部的文件

```
┌──(gyb㉿bogon)-[~/tool/GitHack]
└─$ wget -r http://171.80.2.169:17371/.git
--2026-03-31 21:17:39--  http://171.80.2.169:17371/.git
正在连接 171.80.2.169:17371... 已连接。
已发出 HTTP 请求，正在等待回应 ... 301 Moved Permanently
位置：http://171.80.2.169:17371/.git/ [跟随至新的 URL]
--2026-03-31 21:17:39--  http://171.80.2.169:17371/.git/
再次使用存在的到 171.80.2.169:17371 的连接。
已发出 HTTP 请求，正在等待回应 ... 200 OK
长度：3080 (3.0K) [text/html]
正在保存至："171.80.2.169:17371/.git"

171.80.2.169:17371 100%[═══════════════════>]   3.01K  --.-KB/s  用时 0s

2026-03-31 21:17:39 (116 MB/s) - 已保存 "171.80.2.169:17371/.git" [3080/3080
])

正在载入 robots.txt；请忽略错误消息。
```

下载完成后去这个目录里面打开终端使用 git reflog 查看历史操作记录

```
┌──(gyb㉿bogon)-[~/tool/GitHack/171.80.2.169:17371]
└─$ git reflog
d256328 (HEAD → master) HEAD@{0}: reset: moving to d25632
13ce8d0 HEAD@{1}: commit: flag is here?
fdce35e HEAD@{2}: reset: moving to fdce35e
e0b8e8e HEAD@{3}: reset: moving to e0b8e
40c6d51 HEAD@{4}: commit: flag is here?
fdce35e HEAD@{5}: commit: flag is here?
d256328 (HEAD → master) HEAD@{6}: commit: flag is here?
e0b8e8e HEAD@{7}: commit (initial): this is index.html
```

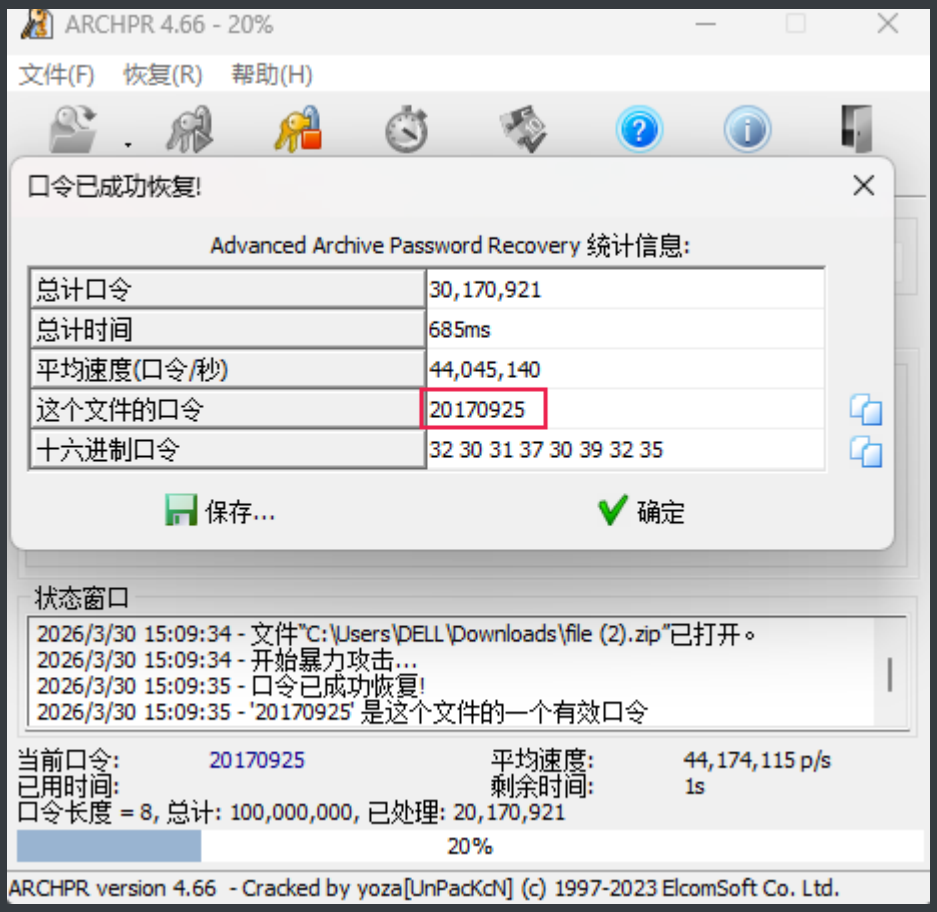
可以看到有一堆的前缀就只能一个个的试了最后40c6d51是正确的地址

```
┌──(gyb㉿bogon)-[~/tool/GitHack/171.80.2.169:17371]
└─$ git show 40c6d51
commit 40c6d51b81775a1590c1b051d9562222e41c4741
Author: vFREE <flag@flag.com>
Date:   Sun Jan 17 20:34:43 2021 +0800

    flag is here?

diff --git a/flag.txt b/flag.txt
index aa6f6dc..726e5d1 100644
--- a/flag.txt
+++ b/flag.txt
@@ -1 +1 @@
-flag{nonono}
+flag{git_is_good_distributed_version_control_system}
```

在被耍了两次后终于是找到了正确的flag：flag{git_is_good_distributed_version_control_system}

5、Crack Zip-Bugku：

下载后发现压缩包需要密码，用ARCHPR对密码进行一下破解，因为不知道密码格式，先尝试一下全数字不断的修改一下范围最后破解成功

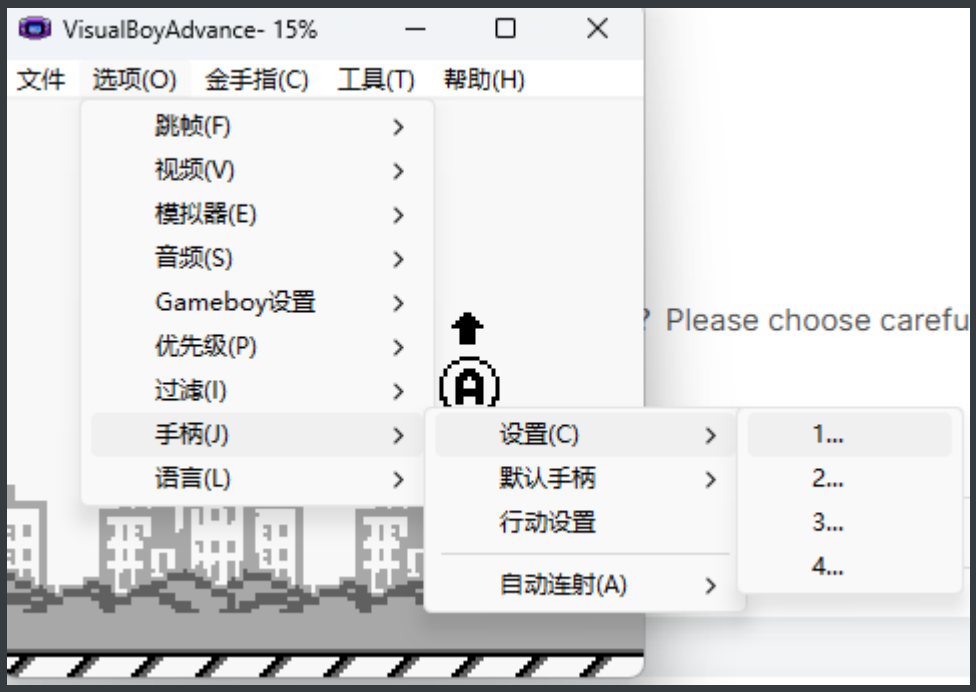


得到密码20170925，输入后解压缩得到flag：Susctf{ec1717de879b19792c77f5edacbb84dc}

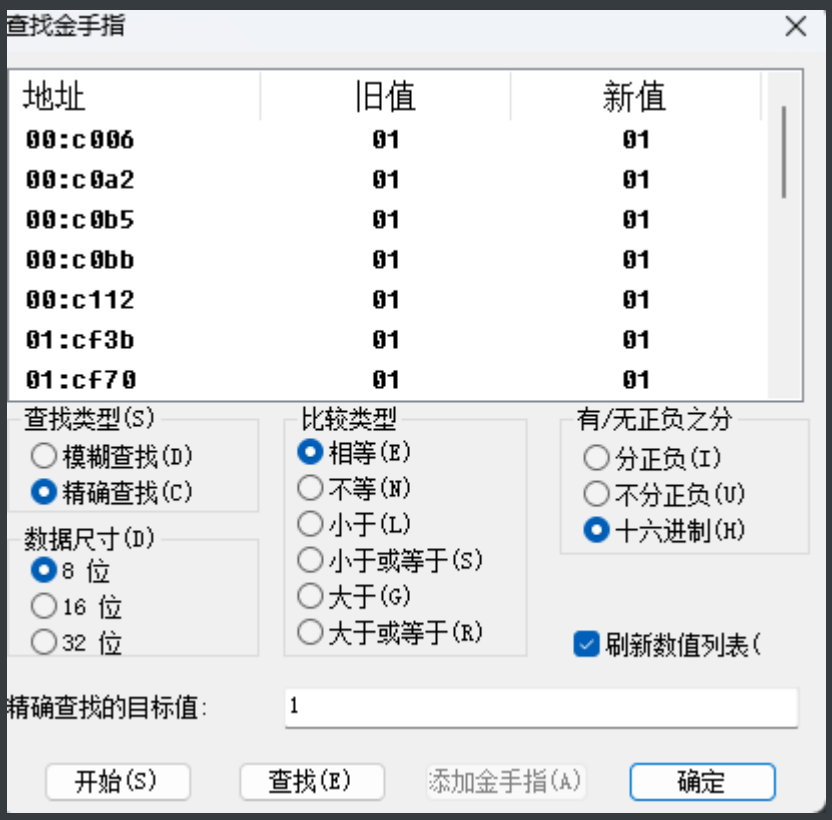
6、鲲or鳗orGame-Bugku：

文件下载下来有一个gb文件和两个音频文件，不知道啥是gb文件搜了一下是一种游戏文件需要用vb模拟器进行打开来游玩，杂项的这种题一般都是要去玩的，下了vb模拟器打开后，什么界面都没提示摸索了半天才发现按enter启动，启动后为第二张图显示，然后说按A但是咋按都没反应

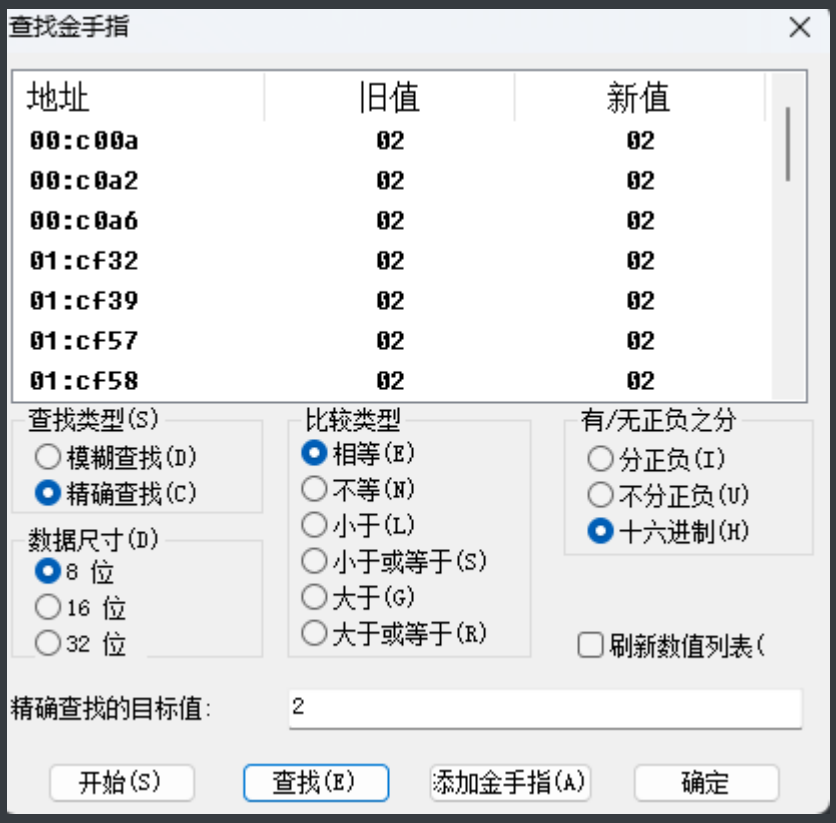
然后再去摸索找到了。我去原来要按z键



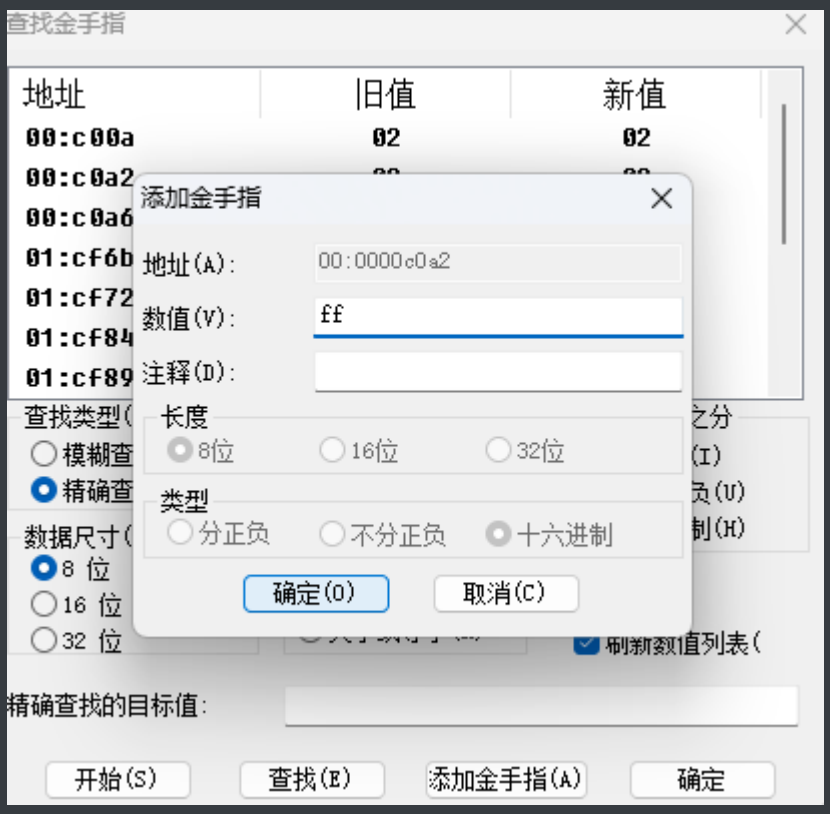
开始玩游戏，这游戏是真的难玩啊，随便跳几下就死了根本找不到任何flag的头绪啊，看了眼wp要通过第二根柱子才能拿到flag，通过第一个时点击查找金手指可以找到一堆地址



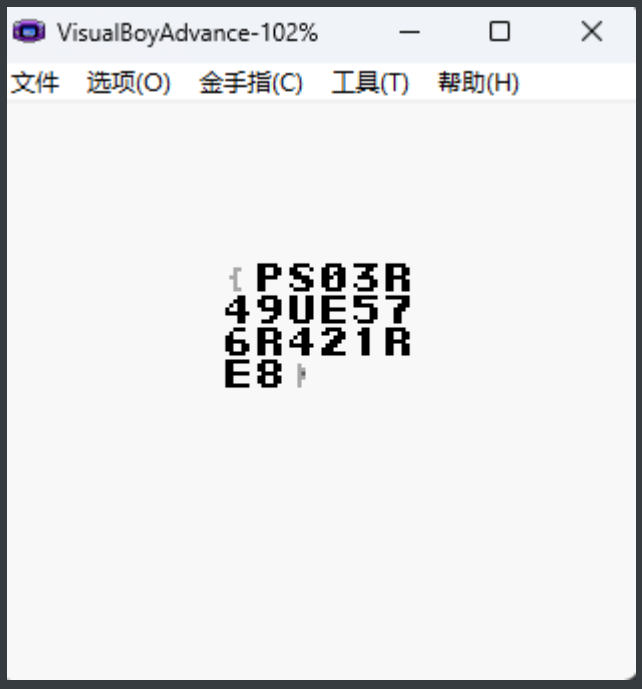
通过第二个柱子后再进行查找



然后就是对地址进行修改，只能一个个的去试，最后发现正确的地址是00：c0a2，去修改这个地址的数值达到最大也就是255，因为是16进制所以修改ff



保存后再进行一把游戏死了后就会直接显示出来flag：flag{PS03R49UE576R421RE8}



只能说这题目真有点抽象来的吧

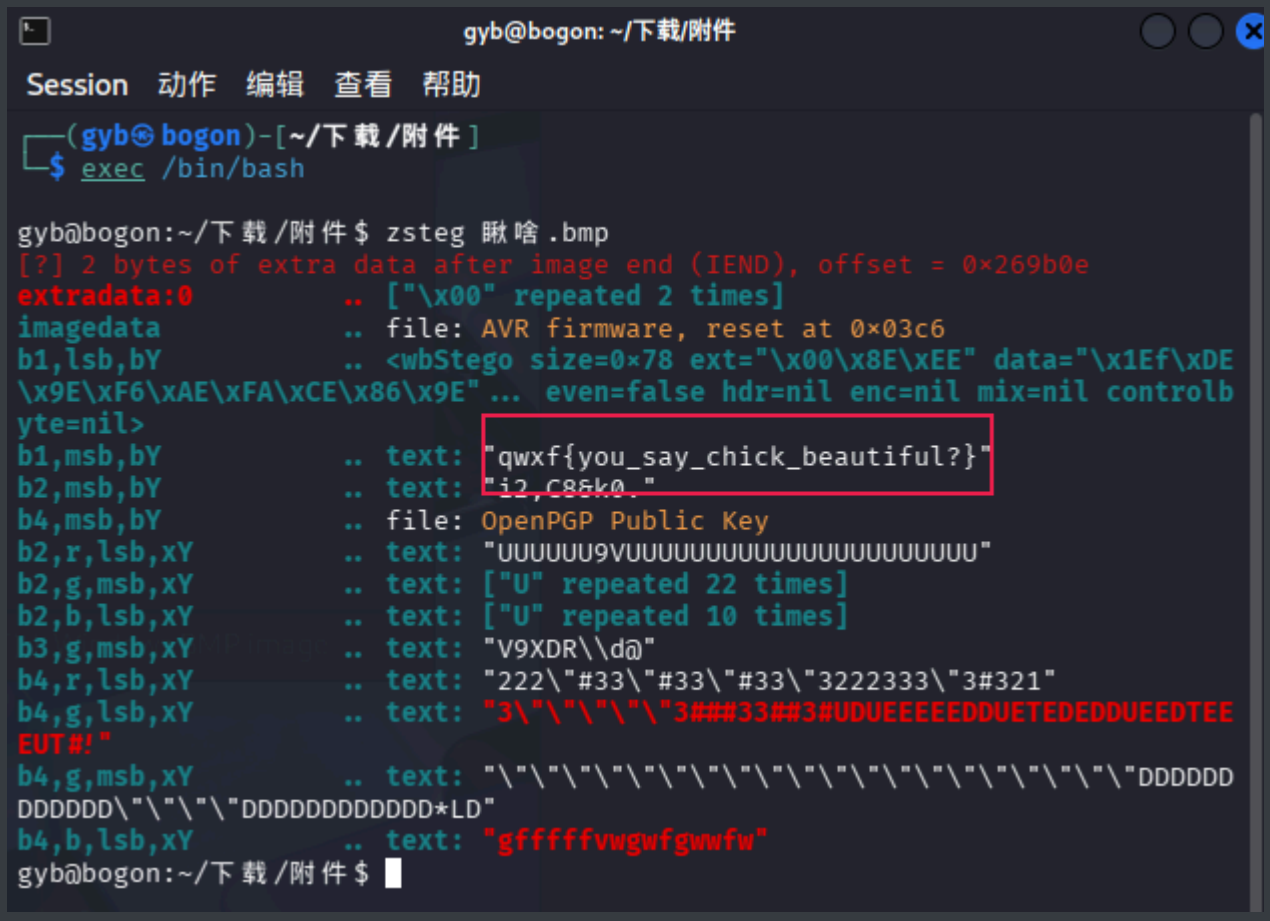
7、强网先锋打野-Bugku：

下载后得到一张bmp的图片，一般这种杂项的题目都是图片隐写的题目，看那个评论区说只能用zsteg来解这道题目，我们在kail上装好zsteg来解这道题目

```
git clone https://github.com/zed-0xff/zsteg
cd zsteg/
gem install zsteg
```

查看一下图片的LSB的信息

```
zsteg 瞅啥.bmp
```



没想到直接把flag给我们了真是太贴心了

8、EasyXor-Bugku:

文件下载后放到IDA里面进行分析找到main函数部分

```
int __fastcall main(int argc, const char **argv, const char **envp)
{
  char v4; // [rsp+Fh] [rbp-91h]
  int v5; // [rsp+10h] [rbp-90h]
  int i; // [rsp+14h] [rbp-8Ch]
  int v7; // [rsp+18h] [rbp-88h]
  _DWORD v8[24]; // [rsp+20h] [rbp-80h]
  char s[24]; // [rsp+80h] [rbp-20h] BYREF
  unsigned __int64 v10; // [rsp+98h] [rbp-8h]
  v10 = __readfsqword(0x28u);
  v8[0] = 83;
  v8[1] = 116;
  v8[2] = 113;
  v8[3] = 96;
  v8[4] = 112;
  v8[5] = 99;
  v8[6] = 125;
  v8[7] = 78;
  v8[8] = 87;
  v8[9] = 103;
  v8[10] = 57;
  v8[11] = 110;
  v8[12] = 104;
  v8[13] = 82;
  v8[14] = 102;
  v8[15] = 106;
  v8[16] = 113;
  v8[17] = 32;
  v8[18] = 123;
  v8[19] = 125;
  v8[20] = 115;
  v8[21] = 104;
  v5 = 0;
  v4 = 1;
```

```
    puts("Mercy: What do you want to tell me?");
    scanf("%s", s);
    v7 = strlen(s);
    if ( v7 != 22 )
      puts("You Are Defented;");
    for ( i = 0; i < v7; ++i )
    {
      if ( v5 )
      {
        if ( v8[i] != (i ^ s[i]) )
          v4 = 0;
        v5 = 0;
      }
      else
      {
        if ( v8[i] != (i ^ s[i]) )
          v4 = 0;
        v5 = 1;
      }
    }
    if ( v4 )
      puts("Heros never die~Victory~");
    else
      puts("You Are Defented;");
    return 0;
  }
```

这是一个典型的 输入验证 程序，核心逻辑是：程序内置了一个加密后的数组 v8（长度为 22），要求用户输入一个 22 个字符的字符串，通过某种变换与 v8 比较，验证成功输出 "Heros never die~Victory~"，失败输出 "You Are Defented;"。要得到正确的输入 s，需要计算s[i] = v8[i] XOR i，我们写一个python脚本来进行解密即可：

```
v8 = [83,116,113,96,112,99,125,78,87,103,57,110,104,82,102,106,113,32,123,125,115,104]
flag = ''.join([chr(v8[i] ^ i) for i in range(len(v8))])
print(flag)
```

运行后输出flag：Susctf{I_n3ed_hea1ing}

9、强网先锋AD-Bugku:

下载文件后放进IDA里面进行分析找到main函数里面发现有一串base的编码

```
__int64 __fastcall main(int a1, char **a2, char **a3)
{
  _BYTE v4[128]; // [rsp+0h] [rbp-150h] BYREF
  _BYTE v5[48]; // [rsp+80h] [rbp-D0h] BYREF
  char v6[32]; // [rsp+B0h] [rbp-A0h] BYREF
  _BYTE v7[112]; // [rsp+D0h] [rbp-80h] BYREF
  _BYTE *v8; // [rsp+140h] [rbp-10h]
  int i; // [rsp+14Ch] [rbp-4h]

  puts("=== Strong Network Pioneer === \n\n");
  __isoc99_scanf("%s", v7);
  strcpy(v6, "qiangwangxianfengjiushini");
  qmemcpy(v5, "ZmxhZ3ttYWZha3VhaWxhaXFpYW5kYW9ifQ==", 36);
  v8 = v7;
  sub_4005B7(v7, v4);
  for ( i = 0; i <= 44; ++i )
  {
    if ( (char)v4[i] != (unsigned __int8)v5[i] )
    {
      puts("you're not\n");
      return 0LL;
    }
  }
  puts("yes, you are!\n");
  return 0LL;
}
```

放进随波逐流里面进行解密就可以直接得到flag：flag{mafakuailaiqiandaob}

```
一键解码:        |解码结果 (注：在线解密密码不参与一键解码)
base64解码:      flag{mafakuailaiqiandaob}
```